39. Combination Sum Medium

Acceptance: 75.9% | LeetCode

Given an array of **distinct** integers `candidates` and a target integer `target`, return *a list of all* ***unique combinations*** *of* `candidates` *where the chosen numbers sum to* `target` *.* You may return the combinations in **any order**.

The **same** number may be chosen from `candidates` an **unlimited number of times**. Two combinations are unique if the frequency of at least one of the chosen numbers is different.

The test cases are generated such that the number of unique combinations that sum up to `target` is less than `150` combinations for the given input.

Example 1:

```
**Input:** candidates = [2,3,6,7], target = 7
**Output:** [[2,2,3],[7]]
**Explanation:**
2 and 3 are candidates, and 2 + 2 + 3 = 7. Note that 2 can be used multiple times.
7 is a candidate, and 7 = 7.
These are the only two combinations.
```

Example 2:

```
**Input:** candidates = [2,3,5], target = 8
**Output:** [[2,2,2,2],[2,3,3],[3,5]]
```

Example 3:

```
**Input:** candidates = [2], target = 1
**Output:** []
```

Constraints:

- `1 <= candidates.length <= 30`
- `2 <= candidates[i] <= 40`
- All elements of `candidates` are **distinct**.
- `1 <= target <= 40`

Claude Opus Solution

Combination Sum — Complete Solution

1. Intuition

The key insight is recognizing this as a **backtracking** problem where we build combinations incrementally. Two crucial observations:

- **Unlimited reuse**: Since each number can be used multiple times, when we pick a candidate, we don't move past it—we allow ourselves to pick it again.
- **Avoiding duplicates**: The combinations `[2,2,3]` and `[2,3,2]` are the same. To count each unique combination **once**, we enforce a rule: *we can only pick the current index or indices that come after it*. This guarantees combinations are built in non-decreasing index order, eliminating permutation duplicates.

This is the classic "subset/combination with a `start` index" pattern.

2. Approach (Pseudocode)

```
function combinationSum(candidates, target):
    result = []
    backtrack(start=0, remaining=target, path=[])
    return result

function backtrack(start, remaining, path):
    if remaining == 0:           # exact match found
        add copy of path to result
        return
    if remaining < 0:            # overshot, dead end
        return

    for i from start to end of candidates:
        path.append(candidates[i])              # choose candidate i
        backtrack(i, remaining - candidates[i], path)  # reuse i allowed
        path.pop()                              # undo choice (backtrack)
```

Key detail: We pass `i` (not `i+1`) to allow reusing the same element. We pass `start` to prevent going backwards (which avoids duplicates).

3. Python Solution

```python
from typing import List


class Solution:
    def combinationSum(self, candidates: List[int], target: int) -> List[List[int]]:
        result = []

        def backtrack(start: int, remaining: int, path: List[int]):
            # BASE CASE 1: exact sum reached -> record a copy of current path
            # (must copy, since 'path' is mutated/reused as we backtrack)
            if remaining == 0:
                result.append(path[:])
                return

            # BASE CASE 2: sum overshot -> prune this branch
            if remaining < 0:
                return

            # EXPLORE: try every candidate from 'start' onward.
            # Starting at 'start' (not 0) ensures non-decreasing order of indices,
            # which prevents duplicate combinations like [2,3] and [3,2].
            for i in range(start, len(candidates)):
                # CHOOSE: add candidate i to the current path
                path.append(candidates[i])

                # RECURSE: pass 'i' (not 'i+1') so candidates[i] can be reused.
                # Subtract its value from the remaining target.
                backtrack(i, remaining - candidates[i], path)

                # UN-CHOOSE (backtrack): remove the last element to try the next one
                path.pop()

        backtrack(0, target, [])
        return result
```

Optional Optimization (Early Pruning with Sorting)

```python
class Solution:
    def combinationSum(self, candidates: List[int], target: int) -> List[List[int]]:
        candidates.sort()  # sort to enable early termination
        result = []

        def backtrack(start: int, remaining: int, path: List[int]):
            if remaining == 0:
                result.append(path[:])
                return

            for i in range(start, len(candidates)):
                # Since sorted, if this candidate already exceeds remaining,
                # all subsequent (larger) candidates will too -> break early.
                if candidates[i] > remaining:
                    break
                path.append(candidates[i])
                backtrack(i, remaining - candidates[i], path)
                path.pop()

        backtrack(0, target, [])
        return result
```

---

4. Complexity Analysis

Time Complexity: `O(N^(T/M + 1))` - `N` = number of candidates, `T` = target, `M` = smallest candidate value. - The recursion tree has a maximum depth of `T/M` (the longest combination uses the smallest value repeatedly). - Each node can branch into up to `N` children, giving a tree size bounded by `O(N^(T/M))`. - Copying each valid path of length up to `T/M` adds another factor, hence the `+1` in the exponent.

Space Complexity: `O(T/M)` - This counts the **recursion stack depth** plus the `path` size, both bounded by the longest combination length `T/M`. - *Note:* This excludes the output list itself, which can hold up to ~150 combinations per the constraints. Including output, it's `O(150 · T/M)`.

---

5. Alternative Approaches

Approach	Idea	Notes
Backtracking (shown)	Build combinations recursively with a `start` index	Cleanest & most common; ideal here
Dynamic Programming	`dp[t]` = list of all combinations summing to `t`, built bottom-up	Works but uses more memory; tricky to avoid duplicates (must enforce non-decreasing order in the build)
BFS / Iterative stack	Replace recursion with an explicit stack of `(start, remaining, path)` states	Same logic, avoids recursion limits

Comparison with the Reference Solution

The reference solution uses `path + [self.cand[i]]` (creating a **new list** each call) instead of `append` / `pop`. Both are correct:

- **Reference style** (`path + [x]`): simpler, no need to copy at the base case, but allocates a new list on every recursive call.
- **My style** (`append` / `pop`): more memory-efficient (one shared `path`), but requires `path[:]` when recording a result.

For interviews, the `append` / `pop` pattern is preferred as it demonstrates the classic backtracking idiom and is generally faster.

Codex Solution

1. Intuition

This is a **backtracking / DFS combination generation** problem.

Key observations:

1. Each candidate can be used **unlimited times**.
2. We need **unique combinations**, not unique permutations.
3. `[2, 2, 3]` and `[3, 2, 2]` are the same combination.
4. To avoid duplicates, we enforce an ordering:
5. Once we choose a candidate at index `i`, future choices can only come from index `i` or later.
6. This allows reusing the same number while preventing going backward and creating permutations.

Example:

```
candidates = [2, 3, 6, 7], target = 7
```

If we choose `2`, we may choose `2` again, then `3`.

But if we choose `3` first, we do not later go back and choose `2`, because that would duplicate `[2, 2, 3]` in a different order.

So the main pattern is:

> Use DFS/backtracking with a `start` index to generate combinations in non-decreasing candidate order.

---

2. Approach

We maintain:

- `path`: current combination being built
- `remaining`: how much sum is still needed
- `start`: the earliest candidate index we are allowed to use

Steps:

1. Sort `candidates`.
2. This helps with pruning.

3. If a candidate is greater than `remaining`, we can stop checking further candidates.
4. Start DFS from index `0` with `remaining = target`.
5. During DFS:
6. If `remaining == 0`, we found a valid combination.
7. If `remaining < 0`, stop.
8. Otherwise, try every candidate from `start` onward.
9. When choosing candidate `candidates[i]`:
10. Add it to `path`.
11. Recurse with the same index `i`, because the same number can be reused.
12. Remove it from `path` after recursion.

Pseudocode

```
function combinationSum(candidates, target):
    sort candidates
    result = []
    path = []

    function dfs(start, remaining):
        if remaining == 0:
            add copy of path to result
            return

        for i from start to len(candidates) - 1:
            value = candidates[i]

            if value > remaining:
                break

            add value to path

            dfs(i, remaining - value)

            remove last value from path

    dfs(0, target)

    return result
```

---

3. Python Solution

```python
from typing import List


class Solution:
    def combinationSum(self, candidates: List[int], target: int) -> List[List[int]]:
        # Sort candidates so that:
        # 1. Combinations are built in non-decreasing order.
        # 2. We can stop early when candidate > remaining target.
        candidates.sort()

        result = []
        path = []

        def backtrack(start: int, remaining: int) -> None:
            """
            start:
                The index from which we are allowed to pick candidates.
                This prevents duplicate permutations.

            remaining:
                The remaining sum we still need to reach target.

            path:
                The current combination being built.
            """
```

```
            # Base case: exact sum reached.
            if remaining == 0:
                # Append a copy because path is modified later.
                result.append(path.copy())
                return

            # Try each candidate starting from 'start'.
            for i in range(start, len(candidates)):
                current = candidates[i]

                # Since candidates is sorted, if current is too large,
                # all later candidates are also too large.
                if current > remaining:
                    break

                # Choose current candidate.
                path.append(current)

                # Recurse.
                #
                # We pass i, not i + 1, because the same candidate
                # may be chosen unlimited times.
                backtrack(i, remaining - current)

                # Undo the choice before trying the next candidate.
                path.pop()

        backtrack(0, target)
        return result
```

---

4. Complexity Analysis

Let:

- `n = len(candidates)`
- `m = min(candidates)`
- `L = target / m`

`L` is the maximum possible length of a combination, because the smallest number can be used at most `target / m` times.

Time Complexity

```
O(n^(target / m))
```

In the worst case:

- The recursion depth can be as large as `target / m`.
- At each level, we may try up to `n` candidates.

So a loose upper bound is:

```
O(n^L), where L = target / min(candidates)
```

Also, whenever we find a valid combination, we copy the current path, which costs up to `O(L)`.

A more output-sensitive way to describe it is:

```
O(number of DFS nodes + number of solutions * L)
```

Sorting costs:

```
O(n log n)
```

So the total is:

```
O(n log n + number of DFS nodes + number of solutions * L)
```

Given the problem constraints, the number of valid combinations is guaranteed to be small.

Space Complexity

Auxiliary recursion/path space:

```
O(target / m)
```

because the deepest recursion path can contain at most `target / min(candidates)` numbers.

Output space:

```
O(number of solutions * L)
```

So including the returned answer:

```
O(number of solutions * L)
```

Excluding the output:

```
O(L)
```

---

5. Alternative Approaches

1. Include / Exclude Backtracking

Instead of looping through candidates, we can write recursion like:

```
At each index:
    either use candidates[index]
    or move to index + 1
```

This is also valid, but the loop-based backtracking version is usually cleaner for this problem.

---

2. Dynamic Programming

We can build combinations for every sum from `0` to `target`.

However, since we need to return all combinations, DP can become memory-heavy and more complicated than backtracking.

Backtracking is preferred here because:

- The constraints are small.
- The output size is limited.
- The problem naturally asks for generating combinations.